alphabet for usernames, but you may need to be more flexible if you need to support international usernames or email addresses as usernames.

**Listing 2.8 Validating inputs**

```
public String createSpace(Request request, Response response)
throws SQLException {
var json = new JSONObject(request.body());
var spaceName = json.getString("name");
if (spaceName.length() > 255) { #A
throw new IllegalArgumentException("space name too long");
}
var owner = json.getString("owner");
if (!owner.matches("[a-zA-Z][a-zA-Z0-9]{1,29}")) { #B
throw new IllegalArgumentException("invalid username: " +
owner);
}
..
}
```

#A Check that the space name is not too long.
#B Here we use a regular expression to ensure the username is valid.

Regular expressions are a useful tool for input validation, as they can succinctly express complex constraints on the input. In this case, the regular expression ensures that the username consists only of alphanumeric characters, doesn’t start with a number, and is between 2 and 30 characters in length. Although powerful, regular expressions can themselves be a source of attack. Some regular expression implementations can be made to consume large amounts of